HIGH-LEVEL DESIGN (HLD)

RAG-Based Customer Support Assistant

with LangGraph & Human-in-the-Loop (HITL)

Internship Project | 2026

1. System Overview

1.1 Problem Definition

Traditional customer support systems rely heavily on human agents to answer repetitive queries. This leads to high operational costs, slow response times, and inconsistent answers. This project addresses this by building an AI-powered customer support assistant that can automatically answer queries from a knowledge base.

1.2 Scope

- Accept natural language queries from users via CLI
- Retrieve relevant information from a PDF knowledge base
- Generate accurate, contextual answers using an LLM
- Route complex or unanswerable queries to a human agent (HITL)
- Use a graph-based workflow for intelligent decision-making

2. Architecture Overview

The system follows a Retrieval-Augmented Generation (RAG) pattern orchestrated by a LangGraph state machine. Below is the high-level data flow:

```
User Query  -->  [LangGraph Workflow]
                        |
              [Retrieve Node]  <-- ChromaDB (Vector Store)
                        |
              [Route Node] ----> Escalate? ---> [HITL Node] --> Human Agent
                        |
              [Answer Node] --> LLM (Groq) --> Final Answer
```

3. Component Description

Component	Role	Technology
Document Loader	Reads PDF and extracts raw text page by page	LangChain PyPDFLoader
Chunker	Splits text into overlapping chunks for precise retrieval	RecursiveCharacterTextSplitter
Embedding Model	Converts text chunks into numerical vectors	HuggingFace all-MiniLM-L6-v2
Vector Store	Stores and searches embeddings by similarity	ChromaDB
Retriever	Fetches top-k similar chunks for a given query	ChromaDB Retriever (k=2)
LLM	Generates natural language answers from context	Groq (LLaMA 3.1 8B)

Graph Engine	Orchestrates nodes and edges in a workflow	LangGraph StateGraph
Routing Layer	Decides between answer node and HITL escalation	Conditional Edge Logic
HITL Module	Escalates to human when confidence is low	CLI Input / Ticket System

4. Data Flow

4.1 Ingestion Pipeline (One-Time)

Step 1: PDF document is loaded using PyPDFLoader, extracting text page by page.

Step 2: Text is split into chunks of ~100 characters with 20-character overlap using RecursiveCharacterTextSplitter.

Step 3: Each chunk is converted to a 384-dimensional embedding vector using HuggingFace MiniLM model.

Step 4: Embeddings are stored in ChromaDB for persistent similarity search.

4.2 Query Lifecycle (Per Request)

Step 1: User submits a natural language question.

Step 2: Retrieve Node: question is embedded and top-2 similar chunks are fetched from ChromaDB.

Step 3: Route Node: checks if context is sufficient and query is not complex. Decides path.

Step 4a: Answer Node: context + question sent to LLM. Answer generated and returned.

Step 4b: HITL Node: question escalated to human agent if routing decides so.

5. Technology Choices

5.1 Why ChromaDB

- Open-source, runs locally — no cloud dependency or cost
- Native Python integration with LangChain
- Fast similarity search using HNSW indexing
- Supports persistent storage across sessions

5.2 Why LangGraph

- Enables graph-based workflow — nodes and conditional edges
- Supports complex multi-step AI pipelines beyond simple chains
- Maintains shared state across all nodes
- Production-ready with support for human-in-the-loop patterns

5.3 Why Groq + LLaMA 3.1

- Groq offers free API tier — suitable for internship projects
- LLaMA 3.1 8B is a capable open-source model for Q&A tasks
- Extremely low latency (< 1 second) due to Groq's LPU hardware

6. Scalability Considerations

Large Documents: Increase chunk_size and use metadata filtering to narrow retrieval scope. Consider hierarchical chunking.

High Query Load: Replace in-memory ChromaDB with a hosted vector DB such as Pinecone or Weaviate. Add caching for repeated queries.

Latency: Pre-load the embedding model at startup. Use async LangGraph invocation for concurrent requests.

Multi-Document: Assign document IDs in metadata and filter by source during retrieval to support multiple knowledge bases.